

```

import React, { forwardRef, useState } from "react";
import { FileText, ImageIcon, AlertCircle } from "lucide-react";
import { clsx, type ClassValue } from "clsx";
import { twMerge } from "tailwind-merge";

// 🌀 LOCAL VANILLA CN UTILITY CORES
export function cn(...inputs: ClassValue[]) {
  return twMerge(clsx(inputs));
}

// ⓘ LOCAL TYPE ISOLATION GATEWAY
export interface ThumbnailProps extends React.HTMLAttributes<HTMLDivElement> {
  src?: string;
  alt?: string;
  fileType?: string;
  size?: "sm" | "md" | "lg";
}

export const Thumbnail = forwardRef<HTMLDivElement, ThumbnailProps>(
  (
    { src, alt = "Preview", fileType, size = "md", className, ...props },
    ref,
  ) => {
    const [imageError, setImageError] = useState(false);
    const [isLoading, setIsLoaded] = useState(false);

    const isImageType = fileType?.startsWith("image/") ?? false;
    const isPdfType = fileType === "application/pdf";

    const extensionMap: Record<string, string> = {
      "application/pdf": "PDF",
      "image/jpeg": "JPG",
      "image/jpg": "JPG",
      "image/png": "PNG",
      "image/gif": "GIF",
      "image/webp": "WEBP",
      "image/svg+xml": "SVG",
      "image/bmp": "BMP",
      "image/tiff": "TIFF",
      "image/x-icon": "ICO",
    };

    const getExtension = (type?: string): string => {
      if (!type) return "FILE";
      if (type.startsWith("image/")) return "IMG";
      return (
        extensionMap[type] ?? type.split("/").pop()?.toUpperCase() ?? "FILE"
      );
    };
  }
);

```

```

const extension = getExtension(fileType);

const sizeClasses: Record<"sm" | "md" | "lg", string> = {
  sm: "w-8 h-8",
  md: "w-10 h-10",
  lg: "w-12 h-12",
};

const iconSizeClasses: Record<"sm" | "md" | "lg", string> = {
  sm: "w-4 h-4",
  md: "w-5 h-5",
  lg: "w-6 h-6",
};

const badgeSizeClasses: Record<"sm" | "md" | "lg", string> = {
  sm: "text-[9px] px-1 py-0.5",
  md: "text-xs px-1.5 py-0.5",
  lg: "text-xs px-2 py-0.5",
};

const containerClasses = cn(
  "relative inline-flex items-center justify-center overflow-hidden",
  "rounded-md",
  "bg-card/90 backdrop-blur",
  "border",
  "border-[color-mix(in_oklch,var(--color-border)_40%,transparent)]",
  "active:scale-95 transition-transform duration-100",
  "hover:shadow-lg transition-shadow duration-200",
  sizeClasses[size],
  className,
);

const imageClasses = cn(
  "absolute inset-0 w-full h-full object-cover",
  "transition-opacity duration-200 ease-out",
  isLoading && !imageError ? "opacity-100" : "opacity-0",
);

const fallbackClasses = cn(
  "relative z-10 flex items-center justify-center w-full h-full",
  "bg-muted/40",
  "text-muted-foreground",
  "transition-opacity duration-200 ease-out",
  !src || imageError || !isImageType ? "opacity-100" : "opacity-0",
);

const handleImageError = () => {
  setImageError(true);
  setIsLoaded(false);
};

```

```

const handleImageLoad = () => {
  setIsLoaded(true);
  setImageError(false);
};

const shouldShowFallback = !src || imageError || !isImageType;

return (
  <div ref={ref} {...props} className={containerClasses}>
    {src && isImageType && !imageError && (
      <img
        src={src}
        alt={alt}
        className={imageClasses}
        onError={handleImageError}
        onLoad={handleImageLoad}
        loading="lazy"
      />
    )}
    <div className={fallbackClasses} aria-hidden={!shouldShowFallback}>
      {isPdfType ? (
        <FileText
          className={cn(iconSizeClasses[size], "text-muted-foreground/80")}
          aria-hidden="true"
        />
      ) : imageError ? (
        <AlertCircle
          className={cn(iconSizeClasses[size], "text-destructive/60")}
          aria-hidden="true"
        />
      ) : (
        <ImageIcon
          className={cn(iconSizeClasses[size], "text-muted-foreground/60")}
          aria-hidden="true"
        />
      )}
    <span
      className={cn(
        "absolute bottom-0 right-0 m-1",
        "rounded-[calc(var(--radius)-2px)]",
        "bg-background/95 backdrop-blur-sm",
        "border",
        "border-[color-mix(in_oklch,var(--color-border)_30%,transparent)]",
        "text-foreground/90 font-medium uppercase tracking-wider",
        "shadow-sm",
        badgeSizeClasses[size],
      )}
      aria-label={`File type: ${extension}`}
    >

```

```
        {extension}
      </span>
    </div>
  </div>
);
},
);

Thumbnail.displayName = "Thumbnail";
```